

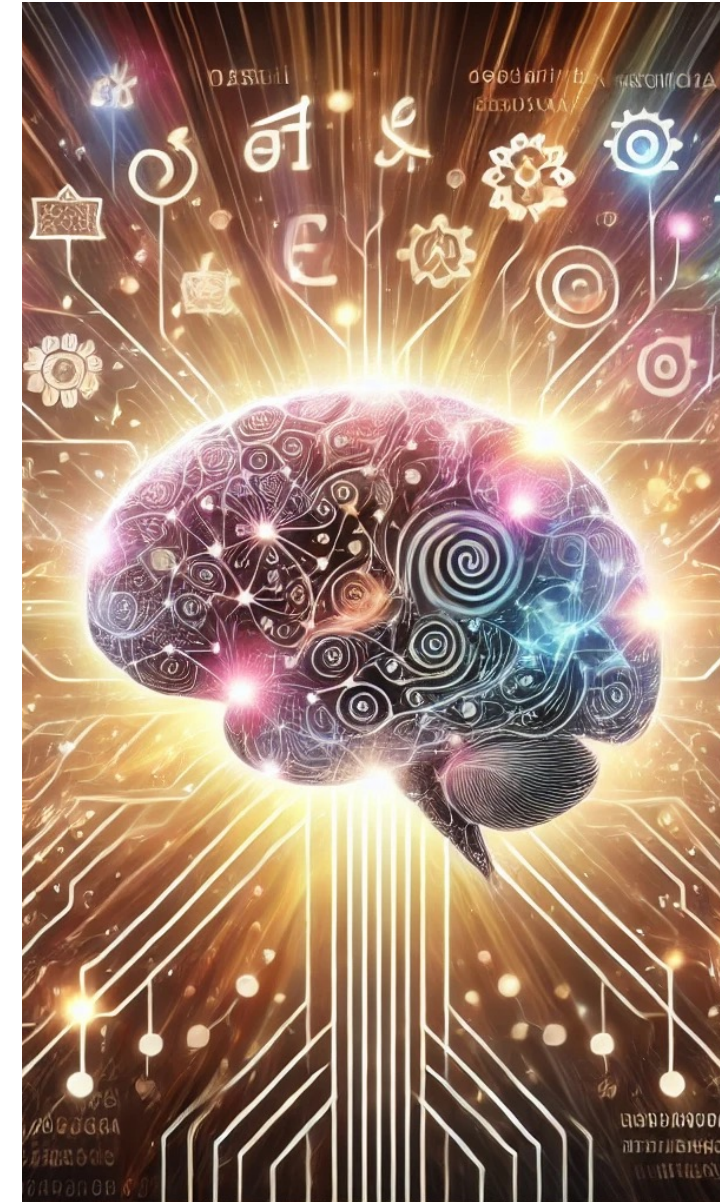
Chapter 12:

Large Language Models

Prachya Boonkwan (Arm)

School of ICT, SIIT,
Thammasat University

prachya@siit.tu.ac.th, kaamanita@gmail.com



Who? Me?

- Nickname: **Arm** (P'/N' Arm, etc.)
- Born: Aug 1981
- Work
 - Researcher at NECTEC 2005-2024
 - Lecturer at SIIT, Thammasat University 2025-now
- Education
 - B.Eng & M.Eng, CPE Kasetsart University
 - Obtained Ministry of Science Scholarship in early 2008
 - Did a PhD in Informatics (AI & Computational Linguistics) at University of Edinburgh, UK from 2008 to 2013 (4.5 years)



Outline

- Introduction
- Zero-shot learning
- Ready-made models and pipelines
- Few-shot learning
- Retrieval-augmented generation

Google Colab

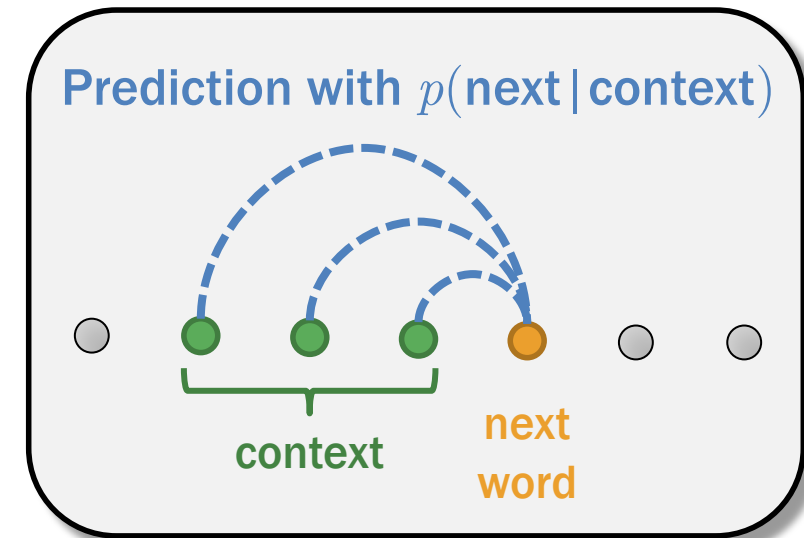
<https://tinyurl.com/4j7jsf9t>

1. Introduction

Large Language Model (LLM)

- Statistical prediction for how strings are produced in a language
- Interpreted as a generative model
 1. Generate the first word w_1
 2. Keep generating the **next word** w_k based on the previous words (a.k.a. **context**) $w_1 \dots w_{k-1}$ until the whole sentence of length N is produced

$$P(w_1 \dots w_N) = p(w_1) \prod_{k=2}^N p(w_k | \overbrace{w_1 \dots w_{k-1}}^{\text{context}})$$



We need at least 1 billion words of text data to train a stable LLM, which learns **word collocations** and **phrase structures**

What is GPT?



อรอุมา สิงห์ศิริ

น้อง

เป็น

สาว

--

What is GPT?



อรอุมา สิงห์ศิริ

น้อง

เป็น

สาว

ขอนแก่น

--

What is GPT?



อรอุมา สิงห์ศิริ

น้อง

เป็น

สาว

ขอนแก่น

ยัง

--

What is GPT?



อรอุมา สิงห์ศิริ

น้อง

เป็น

สาว

ขอนแก่น

ยัง

ป๋

--

What is GPT?



อรอุมา สิงห์ศิริ

น้อง

เป็น

สาว

ขอนแก่น

ยัง

ป๋

เคย

--

What is GPT?



เพลง: สาวอีสานอรัก

อรอุมา สิงห์ศิริ

น้อง

เป็น

สาว

ขอนแก่น

ยัง

บ่

เคย

มี

แฟน

บ้าน

อยู่

แดน

อีสาน

<https://www.youtube.com/watch?v=stLQVIau1ns>

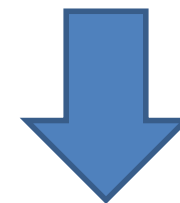
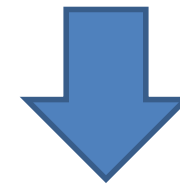
Transformer Model (Vaswani et al., 2016)

- Sequence-to-sequence generation

- Translation:** It learns how to produce a target sequence from a source sequence, given a very large dataset of sequence pairs
- Pros:** It learns **word collocations** and **phrase structures** on the input and output sequences, and associates them cross-lingually in the table of **translation alignments**
- Cons:** It consists of an expansive amount of neuron cells, and the training process can be quite time-consuming

Who	is	the	current	president	of	the	US
-----	----	-----	---------	-----------	----	-----	----

Source: sequence of words (prompt)



The	president	of	the	US	is	Joe	Biden
-----	-----------	----	-----	----	----	-----	-------

Target: sequence of words (response)

Hugging Face Library

- A compendium of Transformer-based pretrained models, pretrained text tokenizers, and prepared datasets
 - Providing an end-to-end pipeline from raw texts to final outputs according to various NLP tasks
 - Offering a platform where developers can share their models, text tokenizers, and datasets to the HuggingFace community with millions of users worldwide



**The AI community
building the future.**

The platform where the machine learning community collaborates on models, datasets, and applications.

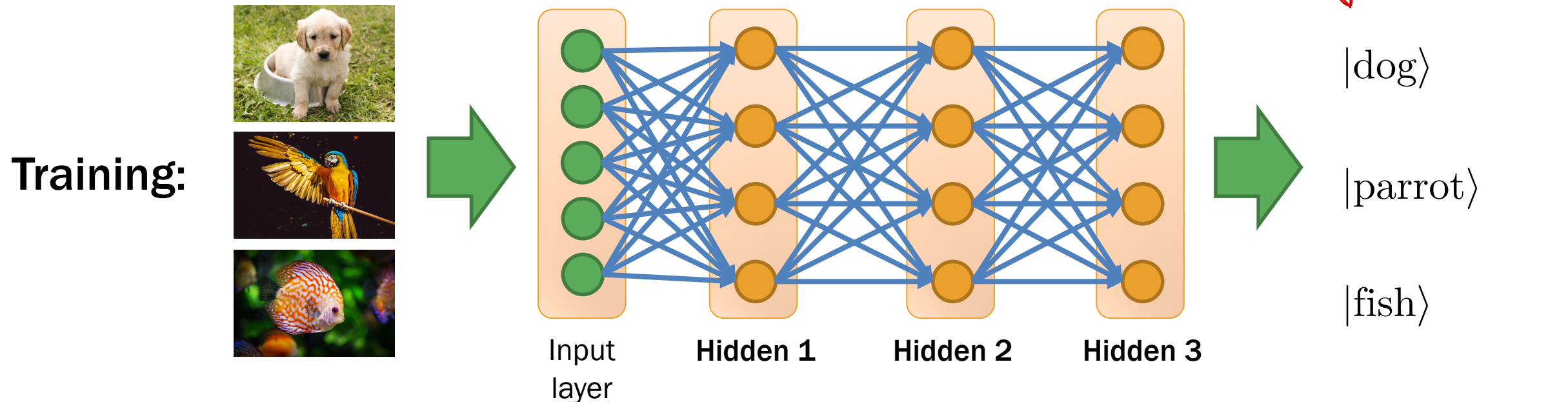
Google Colab

<https://tinyurl.com/4j7jsf9t>

2. Zero-Shot Learning

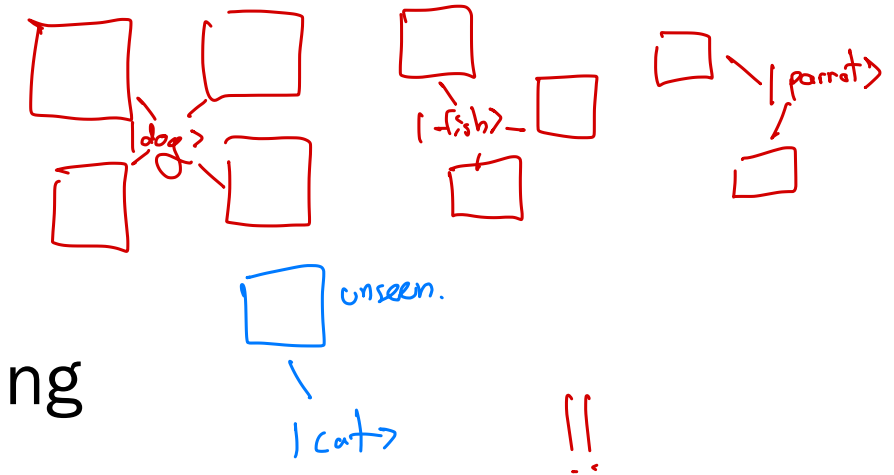
Zero-Shot Learning

- Classifying objects to previously unseen classes, without receiving any specific training for those classes
- Zero shots = zero examples

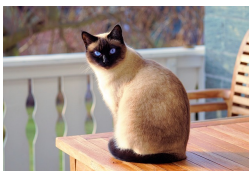


Zero-Shot Learning

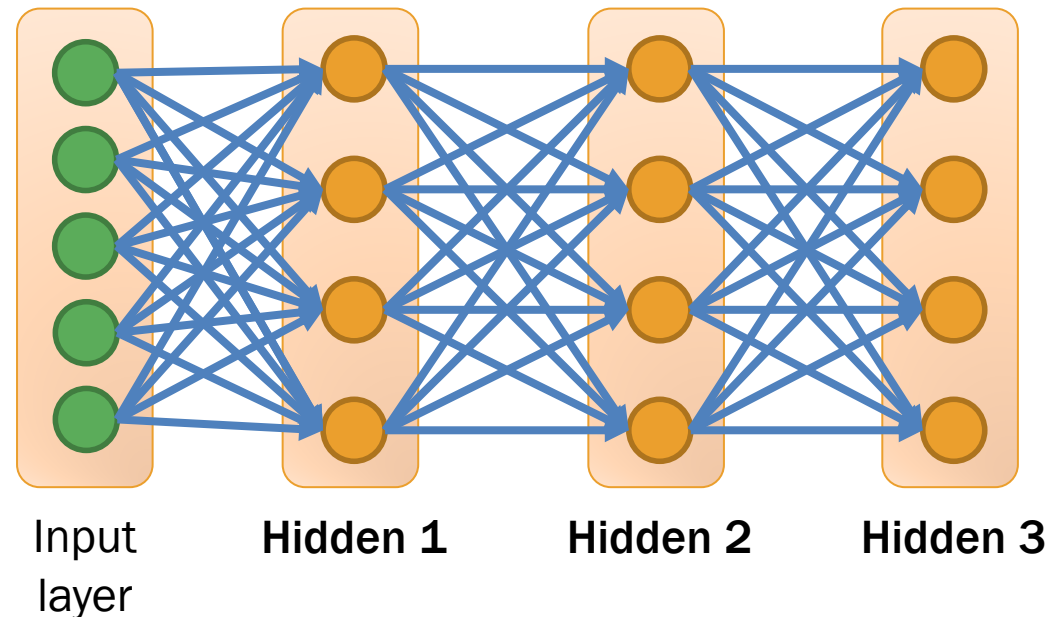
- Classifying objects to previously **unseen classes**, without receiving any specific training for those classes
- Zero shots = zero examples



Testing:



never seen cat images ever.



Still make embed vector for the word cat

$|cat\rangle$

(unseen class)

Zero-Shot Learning with LLM

```
[ ] 1 # We use a pretrained multilingual DeBERTa with the capability of natural language inference.
    2 model = pipeline(
    3     task='zero-shot-classification',
    4     model='MoritzLaurer/mDeBERTa-v3-base-mnli-xnli',
    5     device=get_device()
    6 )
    7
    8 # This is the input text. Note that it can be untokenized!
    9 text = "หมากปริญเผยไม่เคยปิดบังว่ามีแฟนแล้ว"
   10
   11 # This is the label set.
   12 labels = ["politics", "economy", "entertainment", "environment"]
   13
   14 # Classify the text into one of these labels.
   15 output = model(text, labels, multi_label=False)
   16
   17 # Print out the result.
   18 print(f'Input : {text}')
   19 print(f'Output: {output}')
   20 print(f'Most probable class is: {output["labels"][0]} with probability {output["scores"][0]}.'
```

Input : หมากปริญเผยไม่เคยปิดบังว่ามีแฟนแล้ว

Output: {'sequence': 'หมากปริญเผยไม่เคยปิดบังว่ามีแฟนแล้ว', 'labels': ['entertainment', 'politics', 'economy']}

Most probable class is: entertainment with probability 0.5786380767822266.

Zero-Shot Classification

```
[ ] 1 # We use a pretrained DeBERTa with the capability of mask filling.
    2 model = pipeline(
    3     task='fill-mask',
    4     model='MilaNLPProc/deberta-v3-large-mlm-reddit-gab',
    5     device=get_device()
    6 )
    7
    8 # Input text
    9 text = 'Mary has lost her beloved dog, and her eyes are now tearful.'
   10
   11 # The mask is `model.tokenizer.mask_token`.
   12 question = 'The emotion of this text is: ' + model.tokenizer.mask_token + ' .'
   13
   14 # Prompt
   15 prompt = f'"{text}" {question}'
   16 print(f'Prompt: {prompt}')
   17
   18 # The input text and the prompt are put together.
   19 output = model(prompt)
   20
   21 # Print out the result.
   22 for idx, output_entry in enumerate(output):
   23     print(f'Output {idx} = {output_entry}')
```

Prompt: "Mary has lost her beloved dog, and her eyes are now tearful." The emotion of this text is: [MASK] .

Output 0 = {'score': 0.07433608919382095, 'token': 12819, 'token_str': 'sadness', 'sequence': '"Mary has lost

Output 1 = {'score': 0.07278111577033997, 'token': 26817, 'token_str': 'disgust', 'sequence': '"Mary has lost

Output 2 = {'score': 0.02848154865205288, 'token': 5530, 'token_str': 'tears', 'sequence': '"Mary has lost he

Output 3 = {'score': 0.01766805909574032, 'token': 17642, 'token_str': 'sorrow', 'sequence': '"Mary has lost

Output 4 = {'score': 0.014488041400909424, 'token': 10511, 'token_str': 'disappointment', 'sequence': '"Mary

3. Ready-Made Models and Pipelines

Ready-Made Models

▼ NLP

Tasks	Descriptions	Examples
conversational	Conversational agent	PygmalionAI
fill-mask	Mask filling for zero-shot learning	BERT
question-answering	Question answering (extractive)	RoBERTa
table-question-answering	Question answering from a given table	Tapas
text2text-generation	Text generation from a given text	Parrot Paraphraser
text-classification	Document classification and sentiment analysis	DistillBERT
sentiment-analysis	(same as text-classification)	
text-generation	Text generation	GPT-2
token-classification	Sequence prediction (e.g. word segmentation, POS tagging, and NER)	BERT
translation	Machine translation	T5
translation_xx_to_yy	Machine translation from XX to YY (language codes must be small letters)	T5
summarization	Automatic text summarization	BART

▼ Speech Processing

Tasks	Descriptions	Examples
audio-classification	Audio classification	XLSR
automatic-speech-recognition	Automatic speech recognition	Wav2Vec
zero-shot-audio-classification	Zero-shot audio classification	CLIP-HTSAT

▼ Image Processing

Tasks	Descriptions	Examples
depth-estimation	Depth estimation	GLPN
feature-extraction	Image feature extraction for few-shot learning	DINO Vision Transformer
image-classification	Image classification	ResNet-50
image-segmentation	Image segmentation	ClipSeg
mask-generation	Generation of object masks in image processing	SAM Vision Transformer
object-detection	Object detection	YOLOS
video-classification	Video classification	XClip
zero-shot-classification	Zero-shot classification	BART
zero-shot-image-classification	Zero-shot image classification	CLIP Vision Transformer
zero-shot-object-detection	Zero-shot object detection	OWL Vision Transformer

▼ Multimodal Processing

Tasks	Descriptions	Examples
document-question-answering	Question answering from scanned documents	LayoutLM
image-to-text	Caption generation	Vision Transformer + GPT2
visual-question-answering	Visual question answering	ViLT

Example: Sentiment Analysis

```
[ ] 1 # We let the pipeline choose the best model for sentiment analysis.
    2 model = pipeline(
    3     task='sentiment-analysis',
    4     device=get_device()
    5 )
    6
    7 # This is the input text.
    8 text = 'I am promoted as the managing director! Let\'s celebrate tonight.'
    9
   10 # Let's run the model.
   11 output = model(text)
   12
   13 # Here's the result.
   14 print(f'Input : {text}')
   15 print(f'Output: {output}')
```

No model was supplied, defaulted to distilbert-base-uncased-finetuned-sst-2-english
Using a pipeline without specifying a model name and revision in production is not recommended

Downloading (...)lve/main/config.json: 100%  629/629 [00:00<00:00]

Downloading model.safetensors: 100%  268M/268M [00:04<00:00]

Downloading (...)okenizer_config.json: 100%  48.0/48.0 [00:00<00:00]

Downloading (...)solve/main/vocab.txt: 100%  232k/232k [00:00<00:00]

Input : I am promoted as the managing director! Let's celebrate tonight.

Output: [{'label': 'POSITIVE', 'score': 0.9998561143875122}]

4. Few-Shot Learning

Few-Shot Learning

"Pretrained for classification"

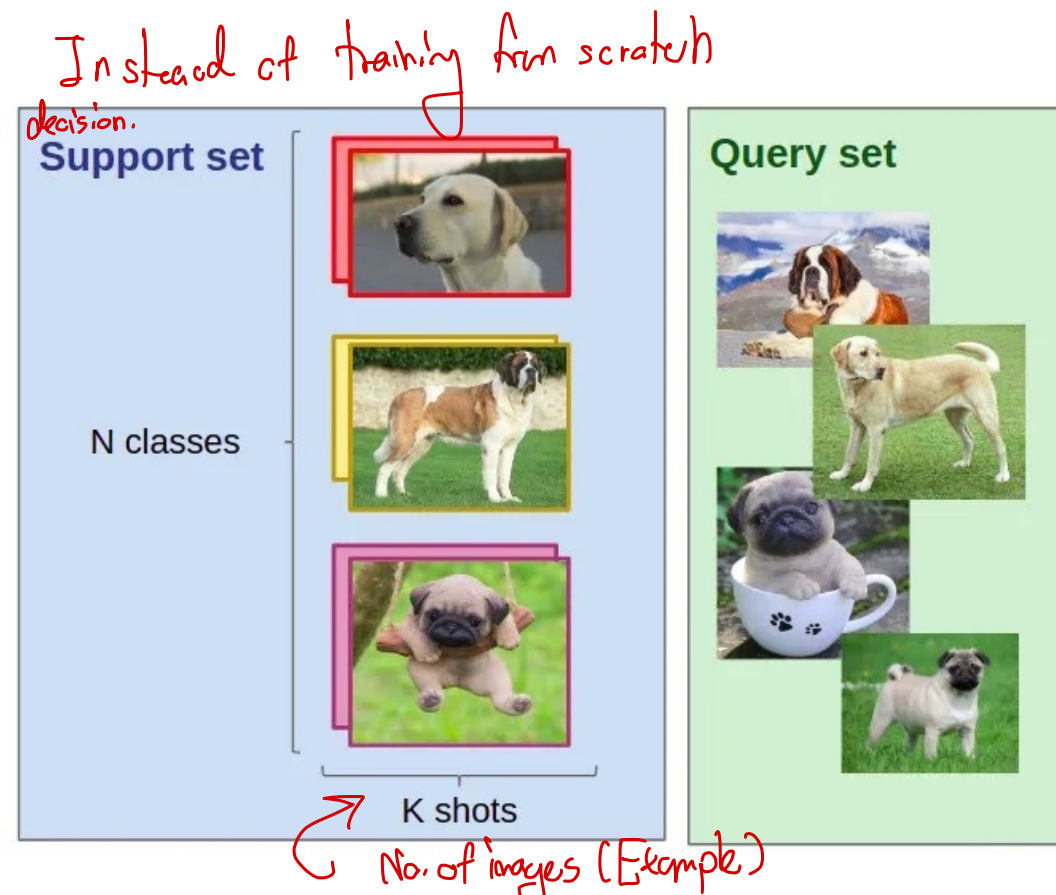
- Learn how to classify objects from previously unseen classes w.r.t. a small example set
- Few shots = few examples

Meta Learning: learning how to build a model

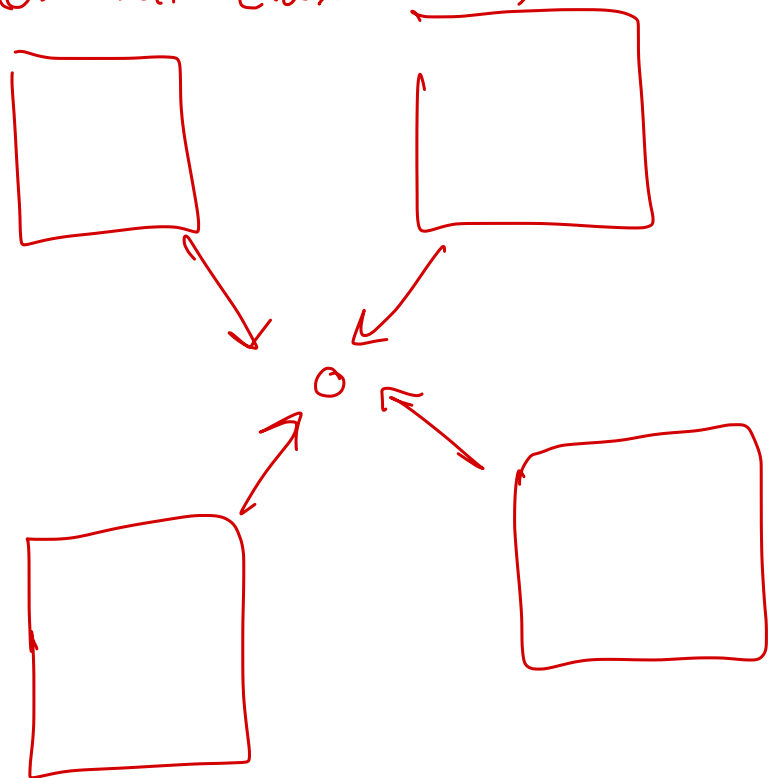
We train a **similarity model** $\text{sim}(\mathbf{x}, \mathbf{x}')$, where $\mathbf{x}$ and $\mathbf{x}'$ are two examples.

- If $\mathbf{x}$ and $\mathbf{x}'$ belong to the same groups, $\text{sim}(\mathbf{x}, \mathbf{x}')$ should be 1.
- If $\mathbf{x}$ and $\mathbf{x}'$ belong to different groups, $\text{sim}(\mathbf{x}, \mathbf{x}')$ should be 0.

The training with backpropagation is done based on the above constraints until convergence. Contrastive learning is employed for training the similarity model, where we use more negative samples with $\text{sim}(\mathbf{x}, \mathbf{x}') = 0$ than positive samples with $\text{sim}(\mathbf{x}, \mathbf{x}') = 1$.



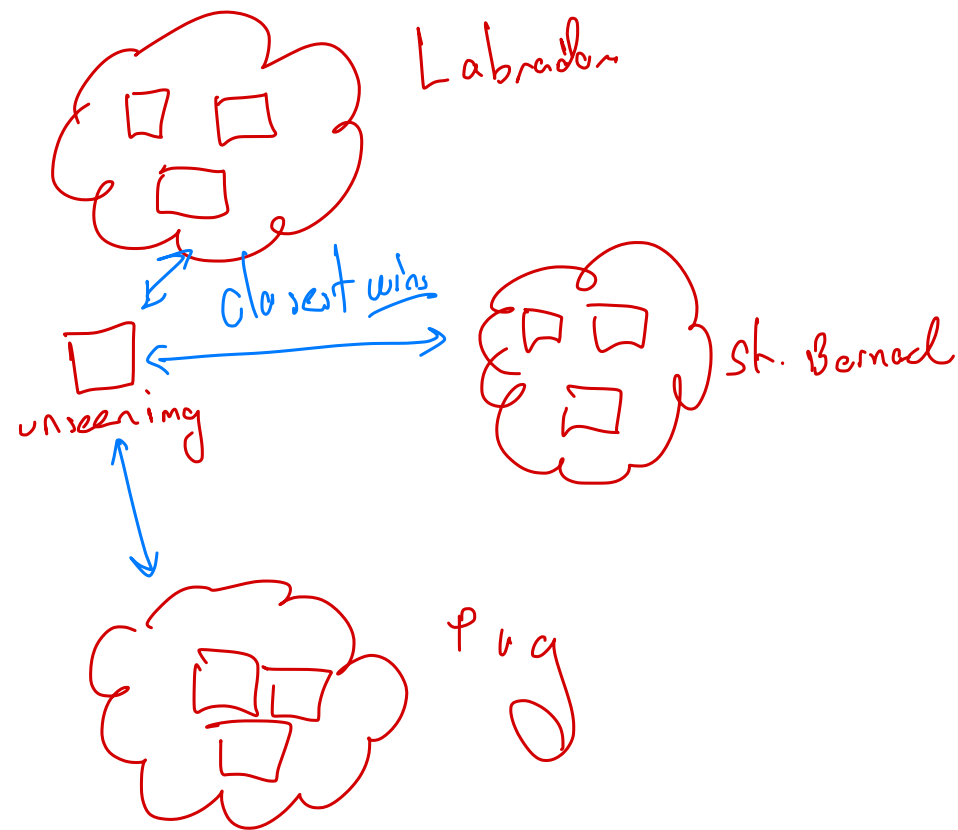
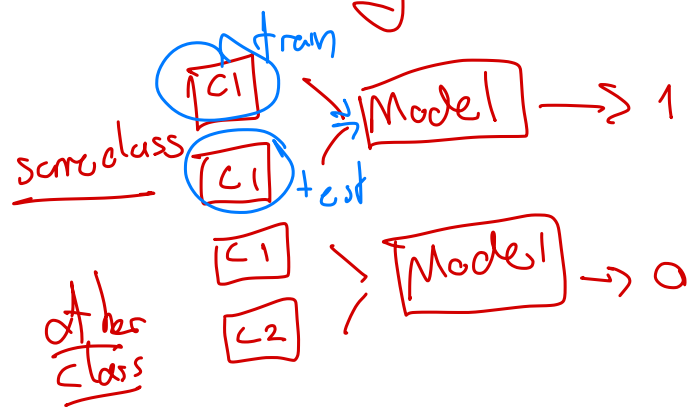
Traditional classification



map directly.

data
give diff
∞

Similarity model



Few-Shot Learning with LLM

```
1 # dataset = load_dataset('StatsGary/socialmedia-abuse')
2 dataset = load_dataset('tweet_eval', 'stance_atheism')
```

```
1 dataset['train'].to_pandas()
```



	text	label
0	@user Bless Almighty God, Almighty Holy Spirit...	1
1	Take away hatred from some people, and you hav...	1
2	I took my troubles to the Lord: I cried out to...	1
3	You can't think by yourself about life and bel...	2
4	RT @user Humanist love to everyone at #100AEUA...	0
...	...	...
456	Pain and setbacks are a part of life. Stay in ...	1
457	Light is not called into Light, its called int...	1
458	Now that the SCOC has ruled Canadians have fre...	2
459	Indigenous would lose more than Madhesi if da ...	0
460	If children weren't religiously indoctrinated ...	1

461 rows x 2 columns

0 none

1 Religion

2 Atheism

Few-Shot Learning with LLM

```
1 # model = SetFitModel.from_pretrained('all-mpnet-base-v2', num_labels=2)
2 model = SetFitModel.from_pretrained('all-mpnet-base-v2', num_labels=3)
3 model.to(get_device())
```

```
1 trainer = SetFitTrainer(
2     model=model,                # Sentence transformer
3     train_dataset=train_ds,      # Training set
4     eval_dataset=eval_ds,       # Test set
5     loss_class=CosineSimilarityLoss, # Similarity measure
6     batch_size=16,
7     num_iterations=20,          # Negative samples for contrastive learning
8     column_mapping={"text": "text", "label": "label"},
9     num_epochs=2,
10    learning_rate=2e-5
11 )
```

```
1 trainer.train() # This step takes 11 minutes on CPU-only training.
```

➡ Applying column mapping to training dataset

Generating Training Pairs: 100%  20/20 [00:00<00:00, 340.02it/s]

***** Running training *****

Num examples = 960

Num epochs = 2

Total optimization steps = 120

Total train batch size = 16

Epoch: 100%  2/2 [28:09<00:00, 844.43s/it]

Iteration: 100%  60/60 [14:08<00:00, 13.67s/it]

Iteration: 100%  60/60 [14:01<00:00, 13.76s/it]

```
[ ] 1 texts = [
2     'Look at that fat chick! Ewww.',
3     'She looks lovely in blonde hair.',
4     'These duffers deserve a low life.'
5 ]
6
7 model.predict(texts)
```

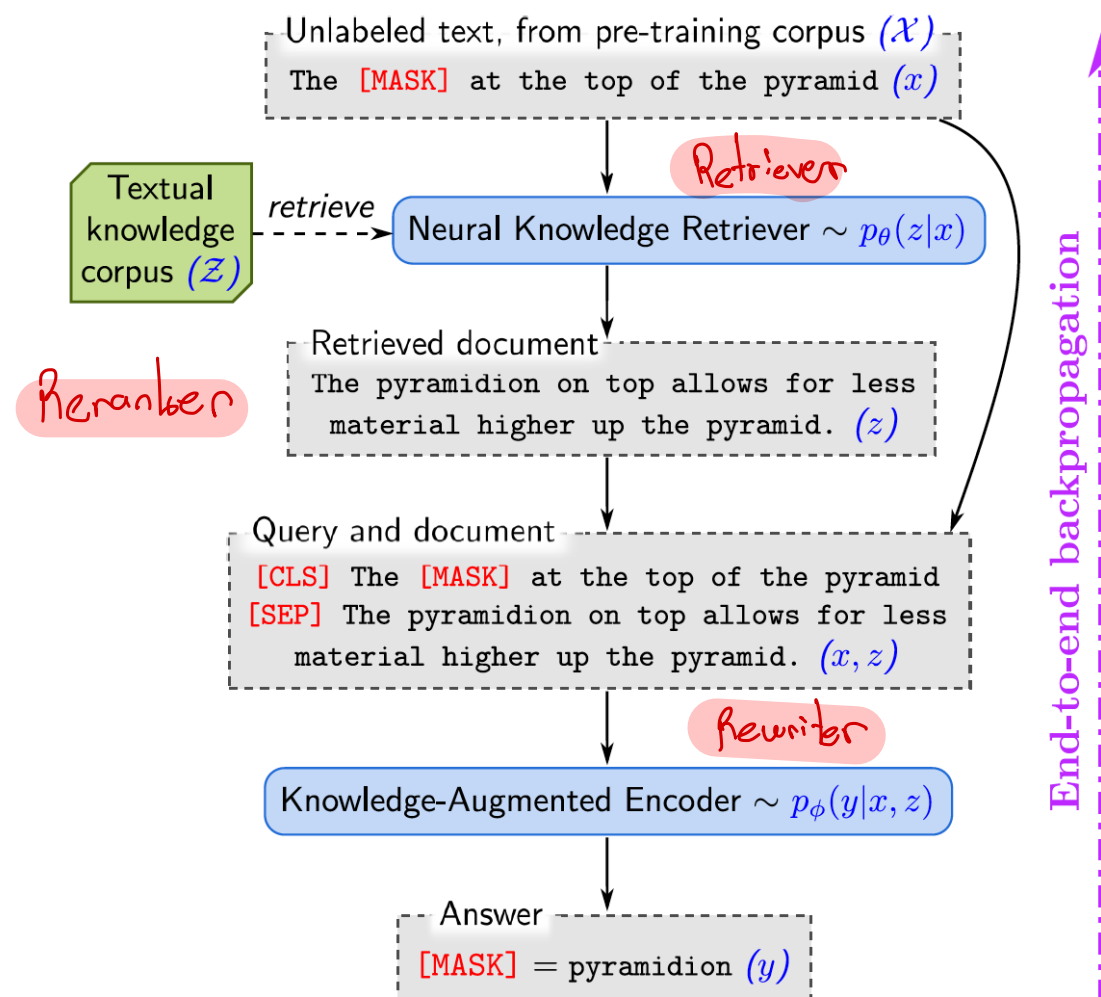
tensor([2, 1, 2])

5. Retrieval-Augmented Generation

Retrieval-Augmented Generation

LLM + Info retrieval
GPT + google (Gemini)

- Enhancing question answering by enriching the query with relevant info
 - Mitigating the hallucination issue in traditional LLMs
 - In-context learning:** simply attaching the query with relevant documents retrieved with information retrieval
 - Vector databases (e.g. FAISS and VectorDB) are used to store internal documents on on-premise servers
 - Advantageous for content access control



RAG in Practical Uses

- Ollama
 - Simple toolkit for deploying a RAG system
- LangChain
 - Generic customizable framework for RAG
 - Easy to deploy and maintain the system
 - <https://www.langchain.com/>
- LocalGPT
 - Based on top of LangChain
 - Quite easy to get started and user-friendly
 - <https://github.com/PromptEngineer/localGPT>

6. Conclusion

Conclusion

- Large language models
- Zero-shot learning
- Ready-made models
- Few-shot learning
- Retrieval-augmented generation

Thank You

prachya@siit.tu.ac.th

kaamanita@gmail.com